CS 3323: Language Interpreter Project

Due: November 20th, 2026 at 11:59pm

1 Overview

The goal of this project is to understand and build an interpreter for a small, OCaml-like, stack-based bytecode language. You will be implementing this interpreter in OCaml, like the previous assignments. The project is broken down into three parts. Part 1 is defined in Section 5 and is worth 100 points. Part 2 is defined in Section 6 and is worth 150 points. Part 3 is defined in Section 7 and is worth 150 points. All parts are due by April 24th 11:59pm, but we recommend you keep to the following time schedule for each part:

1. Part 1 by October 21st @ 11:59pm (100 points)
2. Part 2 by November 4th @ 11:59pm (150 points)
3. Part 3 by November 20th @ 11:59pm (150 points) ***this is the **last day** to submit***

While each part has a deadline, the first two deadlines are "soft" deadlines with only small penalties for missing them. If you submit either a fully working or a partially working solution (e.g. a good faith effort to complete the portion), then you will be still able to earn full marks for that part of the project, and will be allowed to resubmit. If you miss a deadline then you will forfeit 10% of the points for that part of the project.

For example: lets say you submit Part 1 by the deadline and it isn't quite fully finished. You may still resubmit and earn full marks for part 1. However, lets say you miss the deadline for Part 1 by a week. When you submit, fully working or not, the max you can ever earn for this part is 90 Points (or 90%).

You will submit a file named `interpreter.ml` which contains a function, `interpreter`, with the following type signature:

```
val interpreter :  string * string -> unit
```

If your program does not match the type signature, it will not compile on GradeScope and you will receive 0 points. You may, however, have helper functions defined outside of `interpreter`—the grader is only explicitly concerned with the type of `interpreter`.

You must submit a solution for each part. Each part is graded individually, so you may wish, for example, to submit your Part 3 solution for parts 1 and 2. Late submissions will not be accepted and will be given a score of 0. Test cases will also be provided on Canvas for you to test your code locally. These will not be exhaustive, so you are highly encouraged to write your own tests to check your interpreter against all the functionality described in this document.

2 Collaboration

Collaboration among students on this project is not allowed. All code you submit should be your own*. While you may discuss the project with your friends and colleagues these discussions must occur away from the keyboard and away from your computer. Plageurism detection will be used and code found in violation of this policy will face reductions including and potentially beyond a zero on the course project.

There exists one exception to this, the AI use policy for this class. For ***THIS PROJECT ONLY*** you are allowed to utilise an LLM of your choice. There are some restrictions and rules tied to this, of which you should read carefully. You **MUST** submit a written document to the separate submission on Canvas containing the following:

1. Your entire prompt history with the LLM.
2. An explanation for how the AI helped you solve the problem.
3. A justification for how you have still contributed to the code that is being submitted.
4. A discussion of why you used AI in the first place and what it has taught you regarding the specific calls that you made.

A prompt history is defined as a complete copy of each prompt you can provide to an LLM and its corresponding response. Through most LLMs there is an option to share each conversation you have with the LLM via a link. This is completely acceptable. You may have multiple conversations as context frames have limited length. Please either include the links to each conversation in your written submission.

Failure to provide your complete prompt history will result in a score of 0 for the project, even if the code itself is correct. While AI is important to understand in the context of code development, its use needs to be well documented for the purposes of correctness and verification. Additionally, this is a learning environment, and proper use has to be ensured. If you use AI beyond a reasonable scope, e.g. to solve entire components of the project, or without any sort of verification of the correctness of the code, I reserve the right to reduce your grade on the project by the amount that the AI contributed. AI's use needs to be limited to responsible, controlled amounts, as this is a learning environment. Understand that this is the same as how AI is used in industry; while you may use it, you must take ownership of the code that it produces.

Should you decide that you do not want to use AI, this is perfectly fine. You will not receive a penalty for not using it. However, you must still submit a document to the separate submission on Canvas that states that you have not used AI for this project, and that you have written all of the code yourself.

3 Functionality

Given the following function header:
`let interpreter ( (input : string), (output : string )) : unit = ...`

`input` and `output` will be passed in as strings that represent paths to files just like in the Pangram assignment. Your function should write any output your interpreter produces to the file specified by `output`. In the examples below, the input file is read from top to bottom and then each command is executed by your interpreter in the order it was read. It is incredibly useful to read in all of the commands into a list prior to executing them, separating input from the actual interpretation of the commands. The input file can be arbitrarily long. You may find the library function ***String.split_on_char*** to be useful for separating a string into a string list.

input	stack
push 1 quit	1

input	stack
push 6 push 2 div mul quit	:error: 3

input	stack
push 5 neg push 10 push 20 add quit	30 -5

input	stack
push :true: push 7 push 8 push :false: pop sub quit	-1 :true:

input	stack
push 10 push 2 push 8 mul add push 3 sub quit	23

4 Grammar

The following is a context free grammar for the bytecode language you will be implementing. Terminal symbols are identified by `monospace font`, and nonterminal symbols are identified by *italic font*. Anything enclosed in [brackets] denotes an optional character (zero or one occurrences). The form '(set_1 | set_2 | set_n)' means a choice of one character from any one of the n sets. A set enclosed in {braces means zero or more occurrences}.

The set *digit* is the set of digits {0,1,2,3,4,5,6,7,8,9}, *letter* is the set of all characters in the English alphabet (lowercase and uppercase), and *ASCII* is the ASCII character set. The set *simpleASCII* is *ASCII* without quotation marks and the backslash character. Do note that this necessarily implies that escape sequences will not need to be handled in your code.

4.1 Constants

const ::= *int* | *double* | *bool* | *error* | *string* | *name* | *unit*

int ::= [−] *digit* { *digit* }

double ::= [−] *digit* { *digit* } . *digit* { *digit* }

bool ::= `:true:` | `:false:`

error ::= `:error:`

unit ::= `:unit:`

string ::= `"`*simpleASCII* { simpleASCII }`"`

simpleASCII ::= *ASCII* \ {'\', '"'}

name ::= {`_`} *letter* {*letter* | *digit* | `_`}

4.2 Programs

prog ::= *com* {*com*} `quit`

com ::= `push` *const* | `add` | `sub` | `mul` | `div` | `rem` | `neg` | `swap` | `pop` | `cat` | `and` | `or` | `not` | `lessThan` | `equal` | `if` | `bind` | `let` *com* {*com*} `end` | *funBind com* {*com*} [`return`] `funEnd` | `call`

funBind ::= (`fun` | `inOutFun`) $name_1$ $name_2$

5 Part 1: Basic Computation
Suggested Completion Date: October 21st, 2026, at 11:59 pm

Your interpreter should be able to handle the following set of commands:

5.1 push

5.1.1 Pushing Integers to the Stack

push *num*

where *num* is an integer, possibly with a '-' suggesting a negative value. Here '-0' should be regarded as '0'. Entering this expression will simply push *num* onto the stack. For example,

input	stack
push 5	0
push -0	5

If *num* is not an integer and is instead a double, see the instructions in the subsection below.

5.1.2 Pushing Doubles to the Stack

push *num*

where *num* is a double, possibly with a '-' suggesting a negative value. Here '-0.0' should be regarded as '0.0'. Entering this expression will simply push *num* onto the stack. For example,

input	stack
push 5.0	0.0
push -0.0	5.0

If *num* is not a double and is an int, refer to the integer push handling in the section above.

5.1.3 Pushing Strings to the Stack

push *string*

where *string* is a string literal consisting of a sequence of characters enclosed in double quotation marks, as in "this is a string". Executing this command would push the string onto the stack:

input	stack
push "deadpool"	this a string
push "batman"	batman
push "this is a string"	deadpool

Spaces are preserved in the string, i.e. any preceding or trailing whitespace must be kept inside the string that is pushed to the stack:

input	stack
push " deadp ool "	this␣is␣a␣string␣␣
push "this is a string "	␣deadp␣ool␣

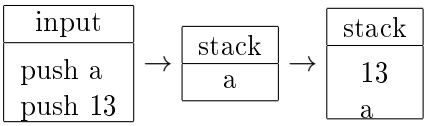
You can assume that the string value would always be legal and not contain quotations or escape sequences within the string itself, i.e. neither double quotes nor backslashes will appear inside a string.

5.2 Pushing Names to the Stack

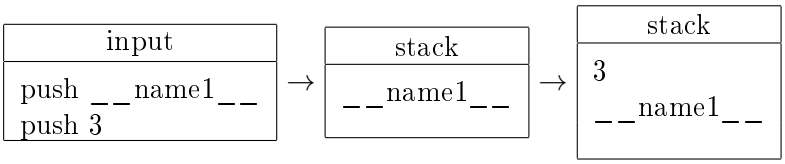
push *name*

where *name* consists of a sequence of letters, digits, and underscores, starting with a letter or underscore.

1. example



2. example



If *name* does not conform to previously mentioned specifications, only push the error literal (`:error:`) onto the stack instead of pushing *name*.

To bind 'a' to the value 13 and `__name1__` to the value 3, we will use 'bind' operation which we will see later (Section 6.7) You can assume that a name will not contain any illegal tokens—no commas, quotation marks, etc. It will always be a sequence of letters, digits, and underscores, starting with a letter (uppercase or lowercase) or an underscore.

5.3 Pushing Booleans to the Stack

push *bool*

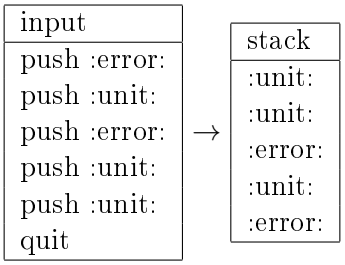
There are two kinds of boolean literals: `:true:` and `:false:`. Your interpreter should push the corresponding value onto the stack. For example,

input	stack
push 5 push :true:	:true: 5

5.4 Pushing error and unit to the Stack

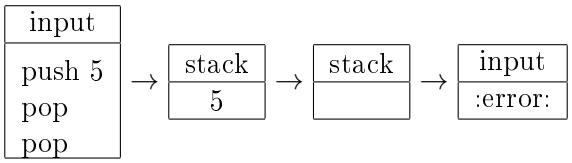
push :error:
push :unit:

Similar with boolean literals, pushing an error literal or unit literal will push `:error:` or `:unit:` onto the stack, respectively.



5.5 pop

The command pop removes the top value from the stack. If the stack is empty, an error literal (`:error:`) will be pushed onto the stack. For example,

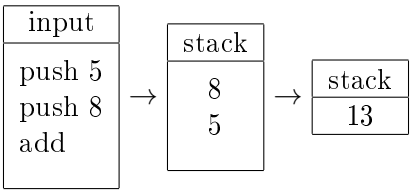


5.6 add

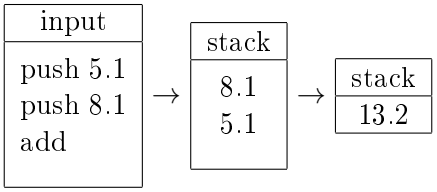
The command add refers to integer OR floating point addition. Since this is a binary operator, it consumes the top two values in the stack, calculates the sum and pushes the result back to the stack. If one of the following cases occurs, which means there is an error, any values popped out from the stack should be pushed back in the same order, then a value `:error:` should also be pushed onto the stack:

- not all top two values are integer numbers OR are double numbers.
- only one value in the stack
- stack is empty

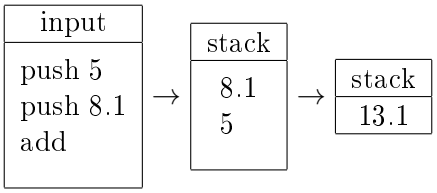
for example, the following non-error case:



or

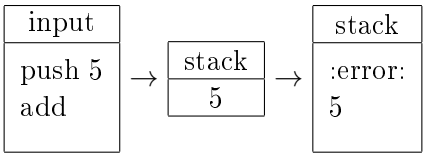


or



Take note that this means that UNLIKE Ocaml, our language can handle automatic numeric conversion. You may want to use float_of_int to convert the type. Note that you will not be expected to convert a double to an int. This operation will only go one way. Type conversion behaviour here will mimmick that of what can be seen in a language like Java. An int can be promoted to a float, but a float cannot be demoted to an int.

Alternately, if there is only one number in the stack and we use add, an error will occur. Then 5 should be pushed back as well as `:error:`

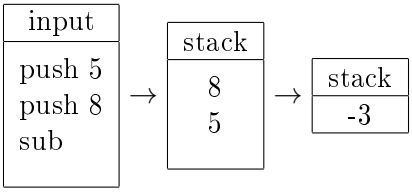


5.7 sub

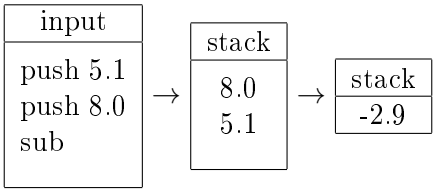
The command sub refers to integer or floating point subtraction. It is a binary operator and works in the following way:

- if top two elements in the stack are integer numbers or double numbers, pop the top element(y) and the next element(x), subtract y from x, and push the result x-y back onto the stack
- if the top two elements in the stack are not all integer or double numbers, push them back in the same order and push `:error:` onto the stack
- if there is only one element in the stack, push it back and push `:error:` onto the stack
- if the stack is empty, push `:error:` onto the stack

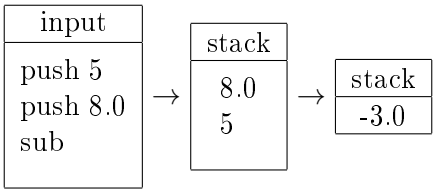
for example, the following non-error case:



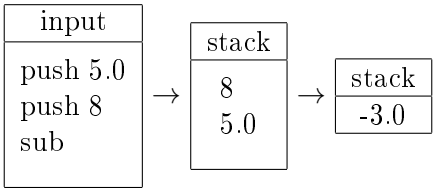
or



or

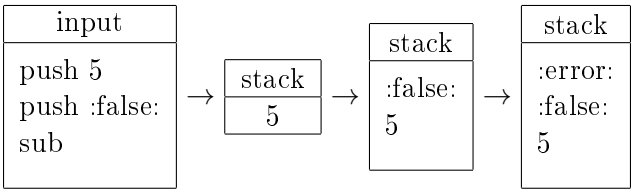


or



Note that with sub, so long as x OR y is a double, then the resultant number on the stack should be double, regardless of whether the computed number needs a double to represent it. You may want to use float_of_int to accomplish this in OCaml.

Alternately, if one of the top two values in the stack is not a numeric number when sub is used, an error will occur. Then 5 and `:false:` should be pushed back as well as `:error:`

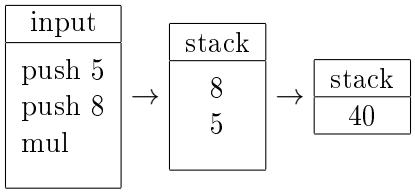


5.8 mul

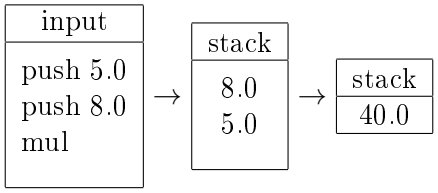
The command mul refers to integer or floating point multiplication. It is a binary operator and works in the following way:

- if top two elements in the stack are integer or double numbers, pop the top element(y) and the next element(x), multiply x by y, and push the result x*y back onto the stack
- if the top two elements in the stack are not all integer or double numbers, push them back in the same order and push `:error:` onto the stack
- if there is only one element in the stack, push it back and push `:error:` onto the stack
- if the stack is empty, push `:error:` onto the stack

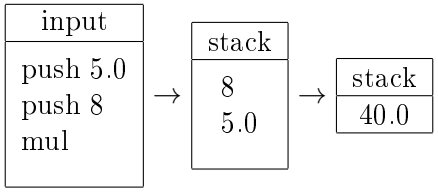
For example, the following non-error case:



or

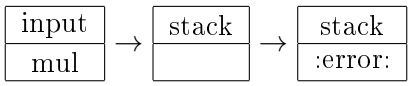


or



Note that once more, this language supports automatic type conversion from int to double when needed but NOT from double to int.

Alternately, if the stack empty when mul is executed, an error will occur and `:error:` should be pushed onto the stack:

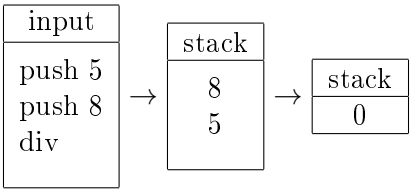


5.9 div

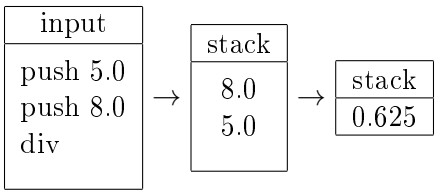
The command div refers to integer or floating point division. It is a binary operator and works in the following way:

- if top two elements in the stack are integer or double numbers, pop the top element(y) and the next element(x), divide x by y, and push the result $\frac{x}{y}$ back onto the stack
- if top two elements in the stack are integer or double numbers but y equals to 0 or 0.0, push them back in the same order and push `:error:` onto the stack
- if the top two elements in the stack are not both integer or double numbers, push them back in the same order and push `:error:` onto the stack
- if there is only one element in the stack, push it back and push `:error:` onto the stack
- if the stack is empty, push `:error:` onto the stack

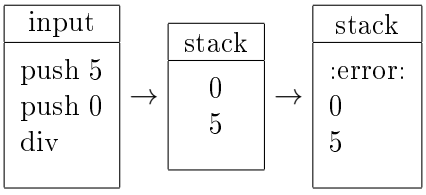
For example, the following non-error case:



or



Alternately, if the top element in the stack equals to 0, there will be an error if div is executed. In such situations 5 and 0 should be pushed back onto the stack as well as `:error:`

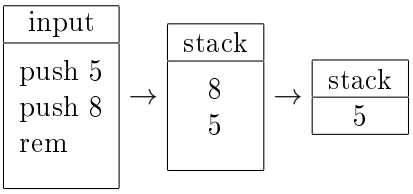


5.10 rem

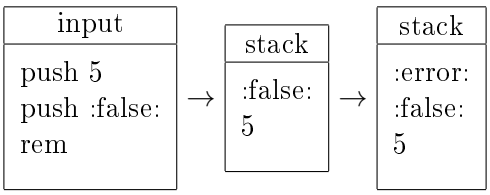
The command rem refers to the remainder of integer division. Rem will ONLY apply to integers in this case, NOT doubles (as this is substantially harder to implement). It is a binary operator and works in the following way:

- if top two elements in the stack are integer numbers, pop the top element(y) and the next element(x), calculate the remainder of $\frac{x}{y}$, and push the result back onto the stack
- if top two elements in the stack are integer numbers but y equals to 0, push them back in the same order and push `:error:` onto the stack
- if the top two elements in the stack are not all integer numbers, push them back and push `:error:` onto the stack
- if there is only one element in the stack, push it back and push `:error:` onto the stack
- if the stack is empty, push `:error:` onto the stack

For example, the following non-error case:



Alternately, if one of the top two elements in the stack is not an integer, an error will occur if rem is executed. If this occurs the top two elements should be pushed back onto the stack as well as `:error:`. For example:

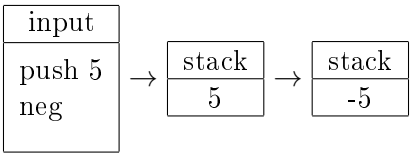


5.11 neg

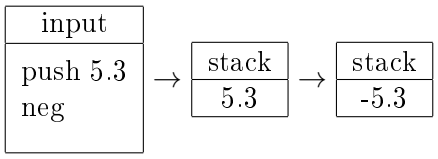
The command neg is to calculate the negation of an integer OR double (negation of 0 should still be 0). It is unary therefore consumes only the top element from the stack, calculate its negation and push the result back. A value `:error:` will be pushed onto the stack if:

- the top element is not an integer, push the top element back and push `:error:`
- the stack is empty, push `:error:` onto the stack

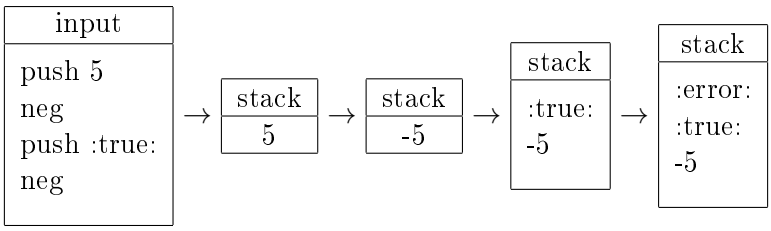
For example, the following non-error case:



or



Alternately, if the value on top of the stack is not an integer, when neg is used, that value should be pushed back onto the stack as well as `:error:`. For example:

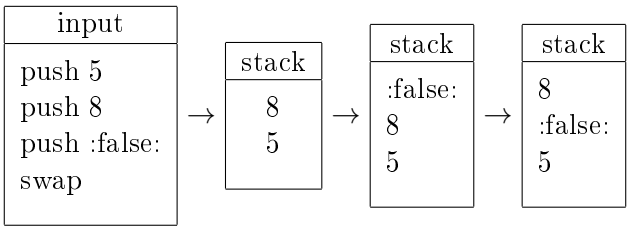


5.12 swap

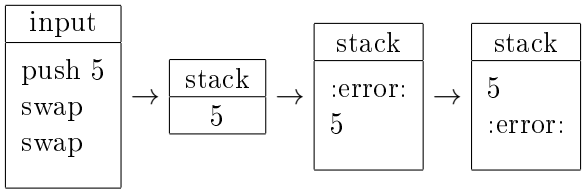
The command swap interchanges the top two elements in the stack, meaning that the first element becomes the second and the second becomes the first. A value `:error:` will be pushed onto the stack if:

- there is only one element in the stack, push the element back and push `:error:`
- the stack is empty, push `:error:` onto the stack

For example, the following non-error case:



Alternately, if there is only one element in the stack when swap is used, an error will occur and `:error:` should be pushed onto the stack. Now we have two elements in the stack (5 and `:error:`), therefore the second swap will interchange the two elements:



5.13 toString

The command toString removes the top element of the stack and converts it to a string. If the stack is empty, the error value is pushed instead. Conversions of values to strings should be done as follows:

- **Integers** shall be rendered as decimal values with no leading zeros. Negative integers should be prefixed with a '-'.
- **Doubles** shall be rendered as decimal values. Negative doubles should be prefixed with a '-'. Doubles with a .0 will be rendered as just '.' . For example, 7.0 would be rendered as '7.' .
- **Booleans** shall be rendered as the values ":true" and ":false:".
- **Error** values shall be rendered as ":error:"
- **Unit** values shall be rendered as ":unit:"
- **String** values should be left unchanged (i.e. surrounding punctuation may not be added)
- **Names** shall be converted to the corresponding string, contents unchanged.
- **Closures** (once introduced in Part 3) shall be rendered as ":fun:"

5.14 println

The println command pops a string off the top of the stack and writes it, followed by a newline, to the output file that is specified as the second argument to the interpreter function. In the case that the top element on the stack is not a string, it should be returned to the stack and an :error: pushed. If the stack is empty, an :error: shall be pushed.

5.15 quit

The command quit causes the interpreter to stop.

6 Part 2: Variables and Scope
Suggested Completion Date: November 4th, 2026, at 11:59 pm

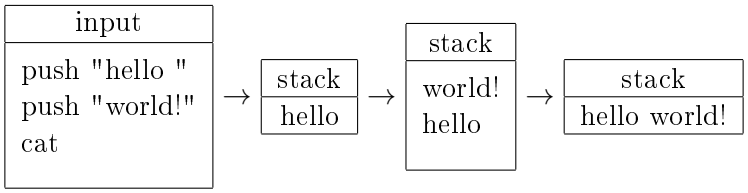
In part 2 of the interpreter you will be expanding the types of computation you will be able to perform, adding support for immutable variables and structures for expressing scope.

6.1 cat

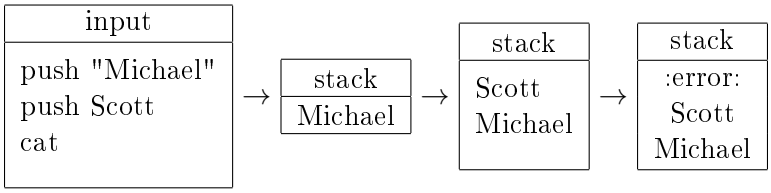
The cat command computes the concatenation of the top two elements in the stack and pushes the result onto the stack. The top two values of the stack — x and y — are popped off and the result is the string y concatenated onto x.
`:error:` will be pushed onto the stack if:

- there is only one element in the stack, push the element back and push `:error:`
- the stack is empty, push `:error:` onto the stack
- if either of the top two elements are not strings, push the elements back onto the stack, and then push `:error:`
  - Hint: Recall that names and strings are different.

For example:



Consider another example:



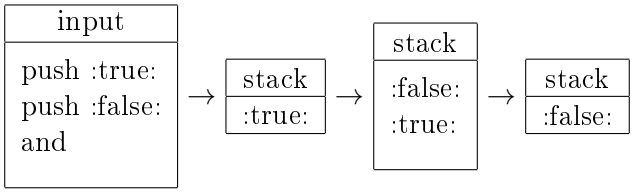
Note that strings can contain spaces, punctuation marks, and other special characters. You may assume that strings only contain ASCII characters and have no escape sequences, e.g. \n and \t.

6.2 and

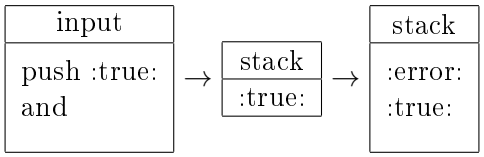
The command and performs the logical conjunction of the top two elements in the stack and pushes the result (a single value) onto the stack.
`:error:` will be pushed onto the stack if:

- there is only one element in the stack, push the element back and push `:error:`
- the stack is empty, push `:error:` onto the stack
- if either of the top two elements are not booleans, push back the elements and push `:error:`

For example:



Consider another example:

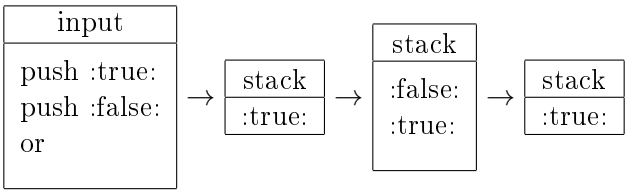


6.3 or

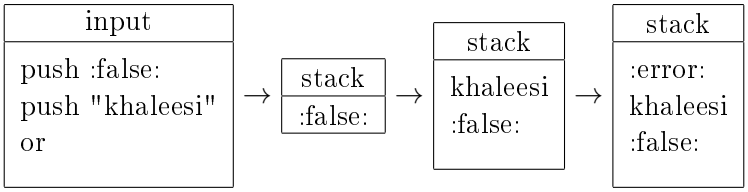
The command or performs the logical disjunction of the top two elements in the stack and pushes the result (a single value) onto the stack.
`:error:` will be pushed onto the stack if:

- there is only one element in the stack, push the element back and push `:error:`
- the stack is empty, push `:error:` onto the stack
- if either of the top two elements are not booleans, push back the elements and push `:error:`

For example:



Consider another example:

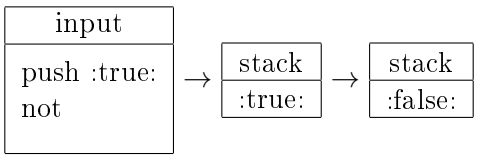


6.4 not

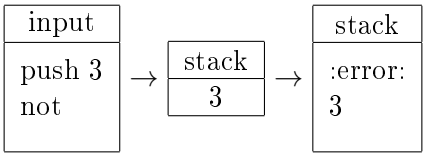
The command not performs the logical negation of the top element in the stack and pushes the result (a single value) onto the stack. Since the operator is unary, it only consumes the top value from the stack. The `:error:` value will be pushed onto the stack if:

- the stack is empty, push `:error:` onto the stack
- if the top element is not a boolean, push back the element and push `:error:`

For example:



Consider another example:

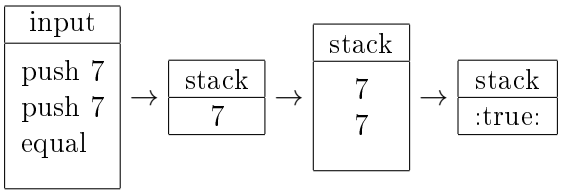


6.5 equal

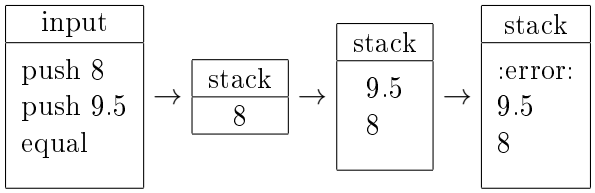
The command equal refers to numeric equality (so you are not supporting string comparisons). The command should handle both Integer as well as floating point values. However, an Integer and a Double are not to be treated the same. For example, 7 and 7.0 are NOT comparable. This operator consumes the top two values on the stack and pushes the result (a single boolean value) onto the stack. The `:error:` value will be pushed onto the stack if:

- there is only one element in the stack, push the element back and push `:error:`
- the stack is empty, push `:error:` onto the stack
- if either of the top two elements are not both Integers OR are not both Doubles, push back the elements and push `:error:`

For example:



Consider another example:

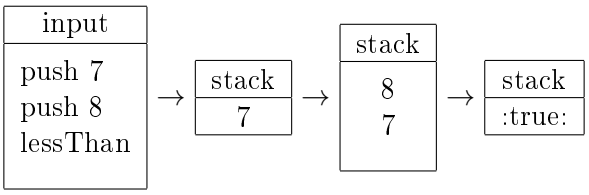


6.6 lessThan

The command lessThan refers to numeric less than ordering. This operator consumes the top two values on the stack and pushes the result (a single boolean value) onto the stack. The command should handle both Integer as well as floating point values. However, an Integer and a Double are not to be treated the same. For example, 7 and 7.0 are NOT comparable. The `:error:` value will be pushed onto the stack if:

- there is only one element in the stack, push the element back and push `:error:`
- the stack is empty, push `:error:` onto the stack
- if the top two elements are not either both Integers or both Doubles, push back the elements and push `:error:`

For example:



6.7 bind

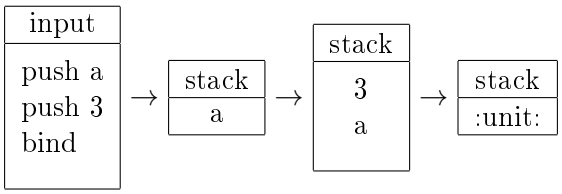
The bind command binds a name to a value. It is evaluated by popping two values from the stack. The second value popped must be a name (see section 5.2 for details on what constitutes a 'name'). The name is bound to the value (the first thing popped off the stack). The value can be any of the following:

- An int
- A double
- A string
- A boolean
- `:unit:`
- The *value* of a name that has been previously bound

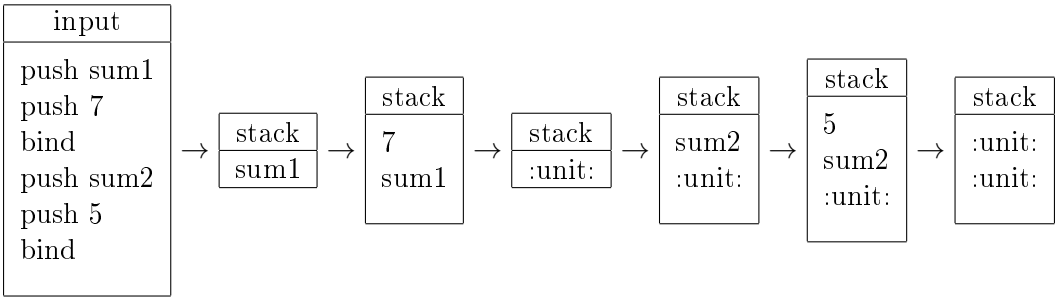
The name value binding is stored in an environment data structure. The result of a bind operation is `:unit:` which is pushed onto the stack. The value `:error:` will be pushed onto the stack if:

- we are trying to bind an identifier to an unbound identifier, in which case all elements popped must be pushed back before pushing `:error:` onto the stack.
- the stack is empty, push `:error:` onto the stack.

6.7.1 Example 1



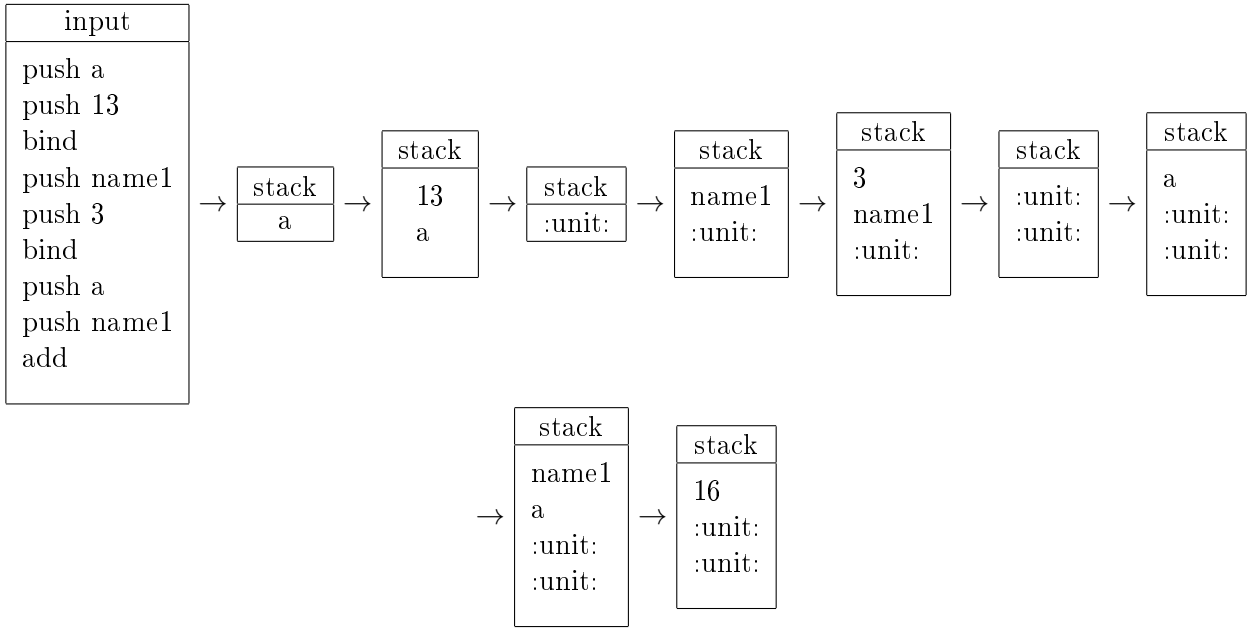
6.7.2 Example 2



You can use bindings to hold values which could be later retrieved and used by functionalities you already implemented. For instance, in the example below, an addition on *a* and *name1* would add 13 + 3 and push the result 16 onto the stack.

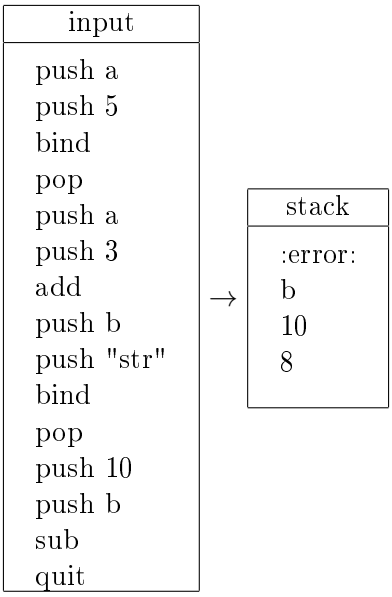
This, in effect, allows names to be in place of proper constants in all the operations we've seen so far. Take for example, when you encounter a name in an add operation, you should retrieve the value the name is bound to, if any. Then if the value the name is bound to has the proper type, you can perform the operation.

6.7.3 Example 3



Notice how we can substitute a constant for a bound name and the commands work as we expect. The idea is that when we encounter names in a command, we resolve the name to the value it's bound to, and then use that value in the operation.

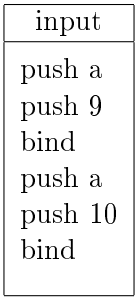
6.8 Example 4



You can see that the add operation completes, because a is bound to an int (5, specifically). The sub operation fails because b is bound to a string, and thus does not type check. While performing operations, if a name has no binding or it evaluates to an improper type, push `:error:` onto the stack, in which case all elements popped must be pushed back before pushing `:error:` onto the stack.

6.9 Example 5

Bindings can be overwritten, for instance:



Here, the second bind updates the value of a to 10.

Common Questions

(a) What values can _name_ be bound to?

name can be bound to ints, booleans, strings, `:unit:` and also previously bound values. For example,

1)

input
push a push :true: bind

would bind a to `:true:`

2)

input
push a push 7.5 bind

would bind `a` to 7.5

3)

input
push b let push a push 7 bind end bind

would bind a to 7 and b to `:unit:`

4)

input
push b push 8 bind push a push b bind

would bind b to 8 and would bind a to the VALUE OF b which is 8.

5)

input
push b push a bind

would result in an `:error:` because you are trying to bind b to an unbound variable a.

(b) How can we bind identifiers to previously bound values?

input
push a push 7 bind push b push a bind

The first bind binds the value of a to 7. The second bind statement would result in the name b getting bound to the VALUE of a—which is 7. This is how we can bind identifiers to previously bound values. Note that we are not binding b to a—we are binding it to the VALUE of a.

(c) Can we have something like this?

input
push a push 15 push a

Yes. In this case a is not bound to any value yet, and the stack contains:

stack
a 15 a

If we had:

input
push a push 15 bind push a

The stack would be:

stack
a :unit:

(d) Can we push the same _name_ twice to the stack? For instance, what would be the result of the following:

input
push a push a quit

This would result in the following stack output:

stack
a a

Yes, you can push the same _name_ twice to the stack. Consider binding it this way:

input
push a push a push 2 bind

This would result in
`:unit:` $\rightarrow$ as a result of binding a to 2
$a \rightarrow$ as a result of pushing the first a to the stack

(e) Output of the following code:

input
push a push 9 bind push a push 10 bind

This would result in the following stack output:

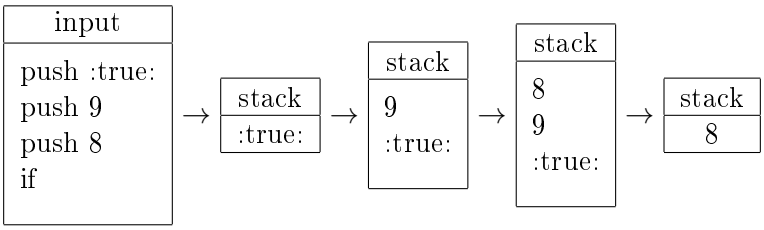
would result in
`:unit:` $\rightarrow$ as a result of second bind
`:unit:` $\rightarrow$ as a result of first bind

6.10 if

The if command pops three values off the stack: x, y and z. The third value popped (z, in this case) must always be a boolean. If z is `:true:`, executing the if command will push x back onto the stack, and if z is `:false:`, executing the if will push y back onto the stack.
`:error:` will be pushed onto the stack if:

- the third value is not a boolean, all elements (x, y, and z) should be pushed back onto the stack before pushing `:error:` onto the stack.
- the stack is empty, push `:error:` onto the stack
- there are less than 3 values on the stack, in which case all elements popped must be pushed back before pushing `:error:` onto the stack.

For example:



Common Questions

(a) What values can 'if' take?

The result of executing a 'if' can be an int or boolean or string or `:error:` or `:unit:`

For instance,

1)

input
push :true: push "oracle" push "jive" if

the result of if would be jive

2)

input
push :false: let push a push 8 bind end push 8.9 if

the result of if would be `:unit:`

(b) What is the result of executing the following:

input
push a push 5 bind pop push :true: push 4 push a if

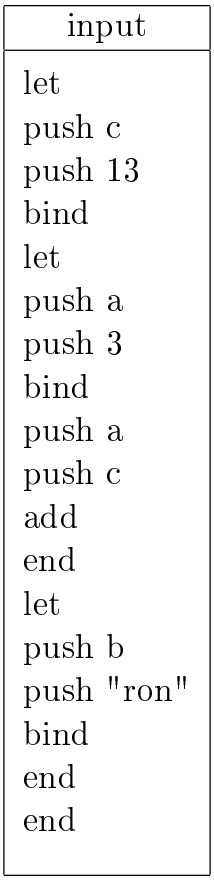
The stack would have a. Although the value of a is bound to 5, we only resolve the name to the value if we need to perform computation. (For 'if', the only value needed for computation is a boolean.)

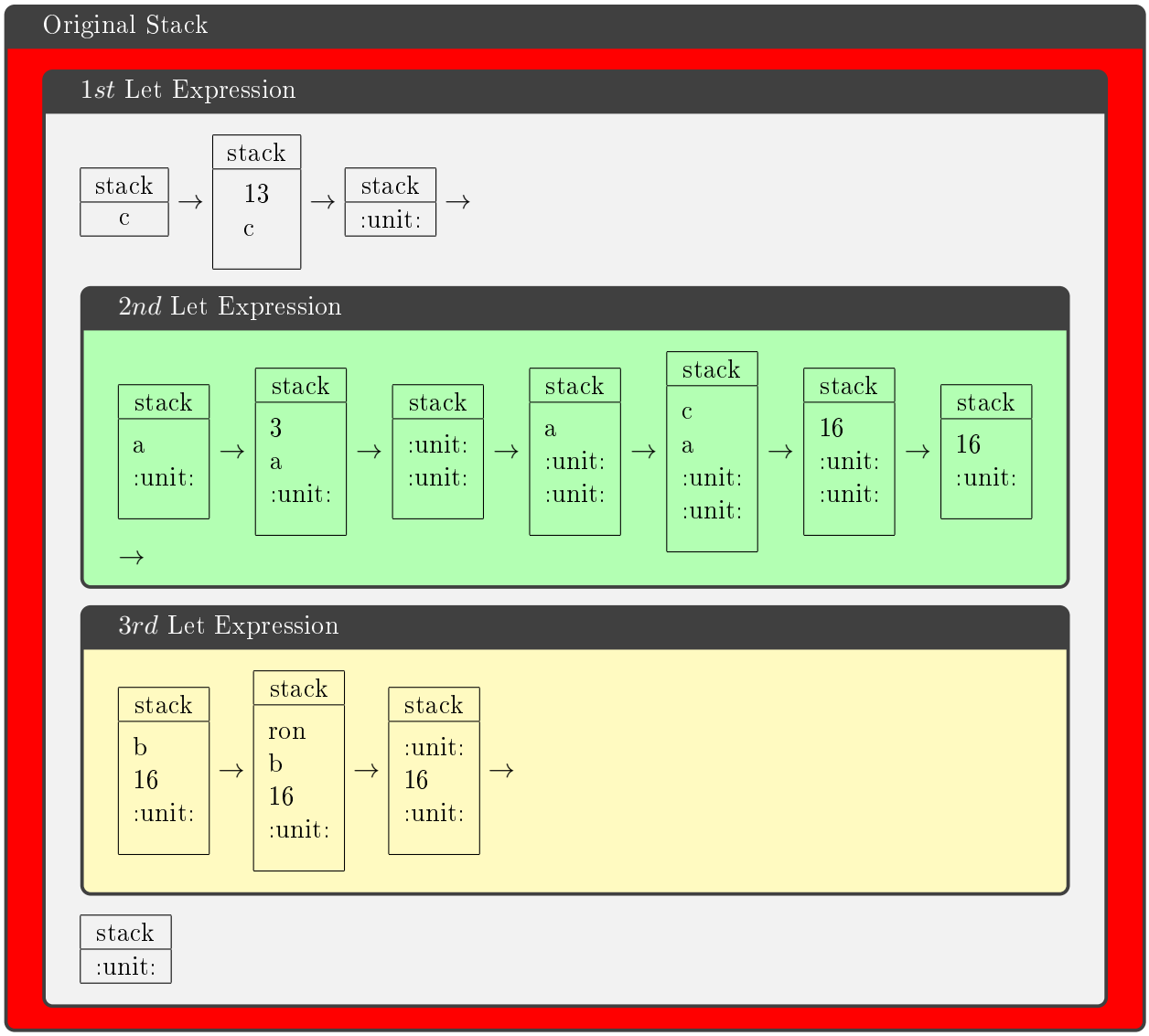
6.11 let...end

let...end limits the scope of variables. "let" marks the beginning of a new environment—which is basically a sequence of bindings. The result of the let...end is the last stack frame of the let. Let...end can contain any number of operations but it will always result in a stack frame that is strictly larger than the stack prior to the let.

Trying to access an element that is not in scope of the let...end block would push `:error:` on the stack. let...end blocks can also be nested.

For example,


input
let
push c
push 13
bind
let
push a
push 3
bind
push a
push c
add
end
let
push b
push "ron"
bind
end
end



In the above example, the first let statement creates an empty environment (environment 1), then the name c is bound to 13. The result of this bind is a `:unit:` on the stack and a name value pair in the environment. The second let statement creates a second empty environment. Name a is bound here. To add a and c, these names are first looked up for their values in the current environment. If the value isn't found in the current environment, it is searched in the outer environment. Here, c is found from environment 1. The sum is pushed to the stack. A third environment is created with one binding 'b'. The second last end is to end the scope of environment 3 and the last end statement is to end the scope of environment 1. You can assume that the stack is left with at least 1 item after the execution of any let...end block.

Common Questions

(a) What would be the output of running the following:

input
push 1 let push 2 push 3 push 4 end push 5

This would result in the stack:

stack
5 4 1

Explanation: After the let...end is executed the last frame is returned—which is why we have 4 on the stack.

(b) What would be the result of executing the following:

input
let push a1 push 7.2 bind end quit

7.2 is pushed to the stack and a1 is bound to `7.2` so, the result would be `7.2`

(c) What would be the output of running the following code:

input
let push 3 push 10 end add quit

The stack output would be:

stack
:error: 10

7 Part 3: Functions Due: November 20th, 2026, at 11:59 pm

7.1 Function declaration and call

fun *name1 name2*

Denotes a function declaration, i.e. the start of a function called *name1*, which has one formal parameter *name2*. The expressions that follow comprise the function body. The function body is terminated with a special keyword funEnd. Note, *name1* and *name2* can be any valid name, but will never be any of the keywords in our language (e.g. add, push, pop, fun, funEnd, etc.). Also the function name and argument name cannot be the same.

funEnd

denotes the end of a function body.

push *funName*
push *arg*
call

Denotes applying the function *funName* to the actual parameter *arg*. When call is evaluated, it will apply the function *funName* to *arg* and pop both *funName* and *arg* from the stack. *arg* can either be a name (this includes function names), an int, a string, a boolean, or `:unit:`.

When the interpreter encounters a function declaration expression it should being construction a closure. A closure will consist of (1) an environment, (2) the code for the function (the expressions between the function declaration and funEnd), and (3) the name of the formal parameter. The value `:unit:` should be pushed to the stack once the function declaration is evaluated and the closure created and bound to the function name in the environment.

1. The environment for the closure will be a copy of the current environment. (Challenge: if you would like to optimize your closure representation you do not need the entire environment, just the bindings of the variables used inside the function that are not defined inside the function and are not the formal parameter).

2. To compute the code for the function, you should copy all the expressions in order starting with the first expressions after the function declaration up to, but not including, the funEnd.

3. In the current environment you should create a binding between the function name and its closure.

When a function is called, you should first check to see if there is a binding in the current environment, which maps *funName* to a closure. If one does not exist, push `:error:` onto the stack. You should then check to see if the current environment contains a binding for *arg* if it is a name instead of a value. If it does not, then you should push `:error:` onto the stack. If *arg* is an `:error:` you should push `:error:` onto the stack.

If both *funName* and *arg* have appropriate bindings, or *arg* is a valid value, then the call to the function can proceed. To do this, push the environment stored in the closure onto the stack. To this environment add a binding between the formal parameter [1] and the value of the actual parameter (i.e. the argument). Note that if *arg* is a name, then it must have a binding in the

[1] you will extract the formal parameter from the closure

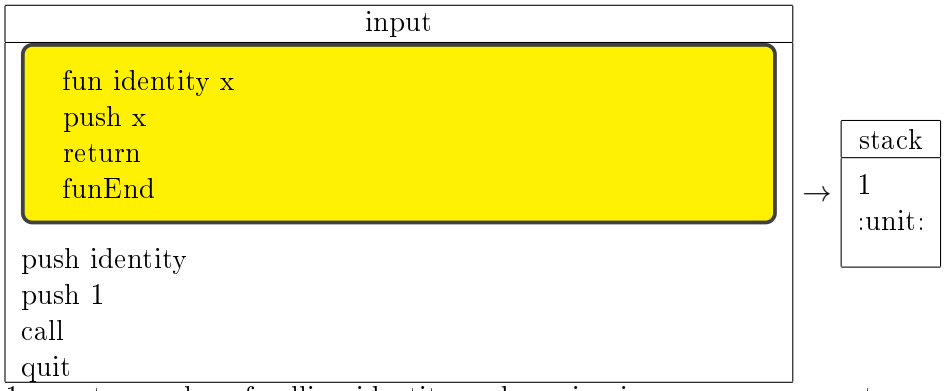
environment at the point of the call [2]. You should then save the current stack and create a new stack that will be used for the execution of the function [3]. Next retrieve the code for the function and begin executing the expressions. The function completes once the last expression in code for the function is executed. When this happens, you should restore the environment to the environment that existed prior to the function call [4]. The stack should also be restored to what the stack was at the point of the call [5]. Once the environment has been restored, execution should resume with the expression that follows the call.

7.2 return

Functions can return values by using a return expression. Since functions themselves are values (a closure), functions can take other functions as arguments and can return functions. When a return expression is evaluated, the function stops execution. When this happens you should restore the environment to the environment that existed prior to the function call, just like if the function completed by executing the last expression in the function's code. The stack should also be restored to what the stack was at the point of the call. Additionally, you should push the last stack frame the function pushed onto the restored stack (the stack at the point of the call).
Please note that background color and indentation is used only to improve readability. Closure would consist of code within colored background.

7.3 Examples

7.3.1 Example 1



1 → return value of calling identity and passing in x as an argument
:unit: → result of declaring identity

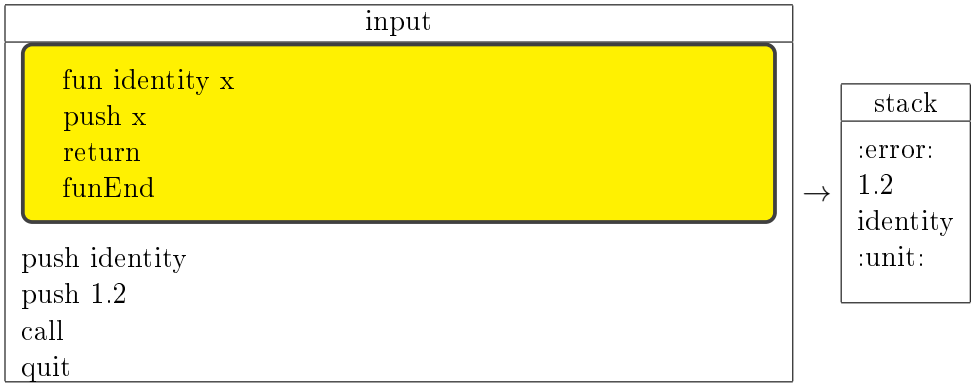
[2] this is the current environment before you push the closure's environment

[3] hint: you may want to implement the stack as a stack of stacks to handled nested function calls and recursion, much like implementing the environment as a stack of maps

[4] hint: if you are implementing your environment as a stack of local environments, this will entail popping off the top environment

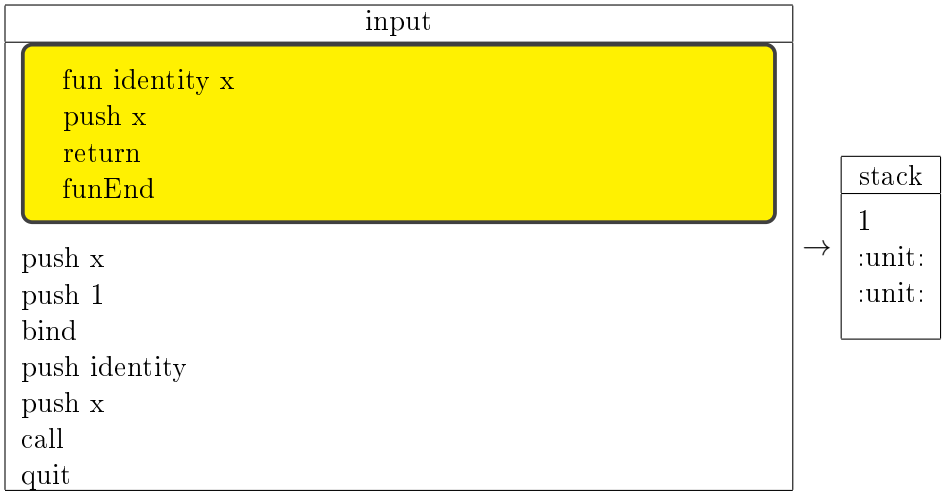
[5] hint: if you implemented your stack as a stack of stacks, this only requires popping off the top stack to restore the stack to what it was prior to the call

7.3.2 Example 2



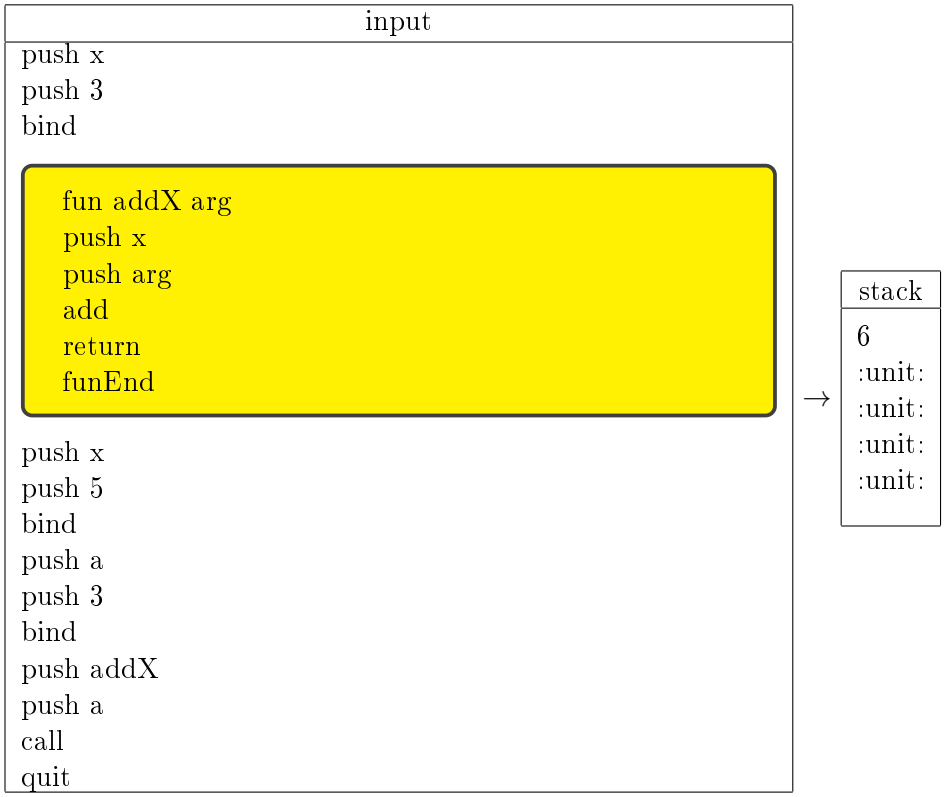
:error: → error as a result of calling a function with error as the actual parameter
1.2 → result of pushing 1.2
identity → push of identity
:unit: → result of declaring identity

7.3.3 Example 3



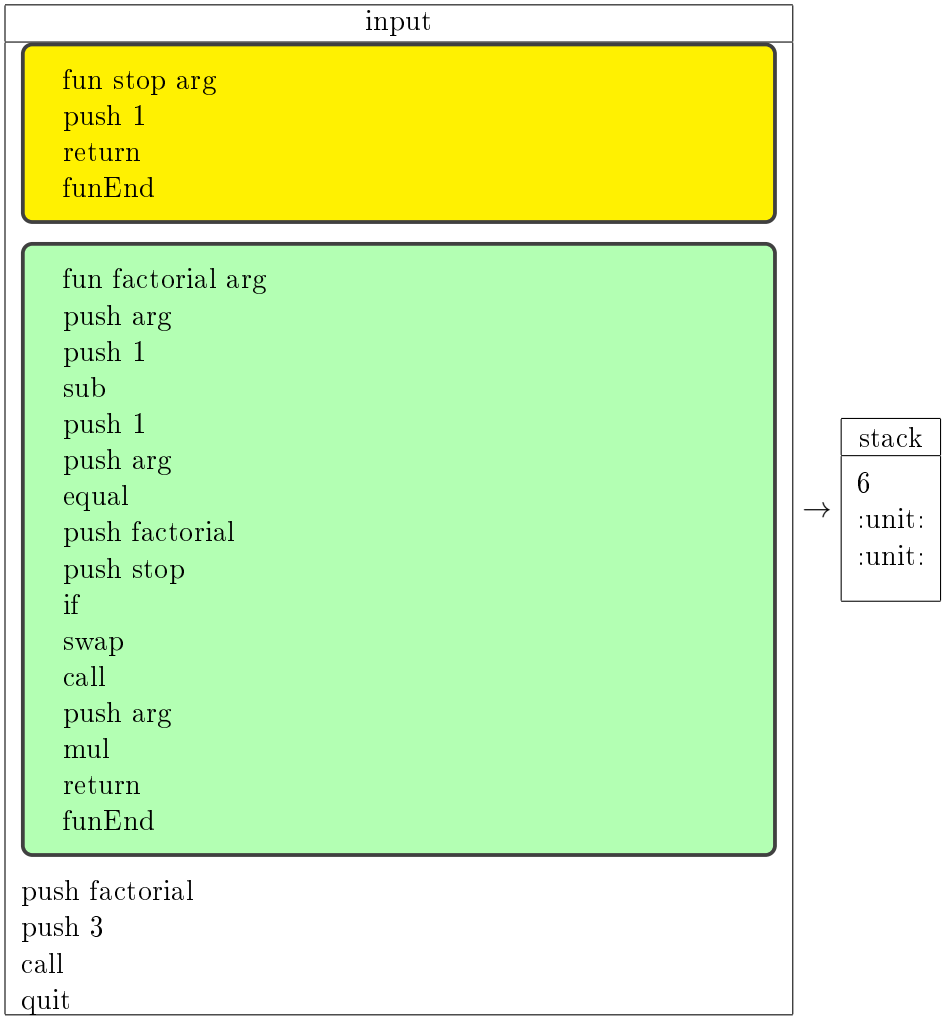
1 → return value of calling identity and passing in x as an argument
:unit: → result of binding x
:unit: → result of declaring identity

7.3.4 Example 4



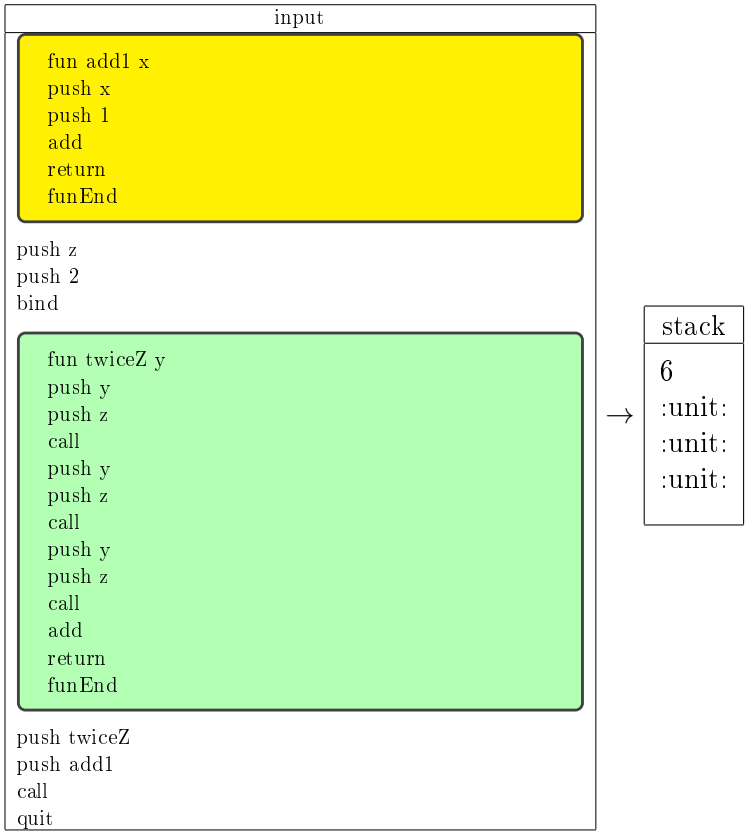
6 → result of function call
:unit: → result of third binding
:unit: → result of second binding
:unit: → result of function declaration
:unit: → result of first binding

7.3.5 Example 5



6 → value returned from factorial
:unit: → declaration of factorial
:unit: → declaration of stop

7.3.6 Example 6

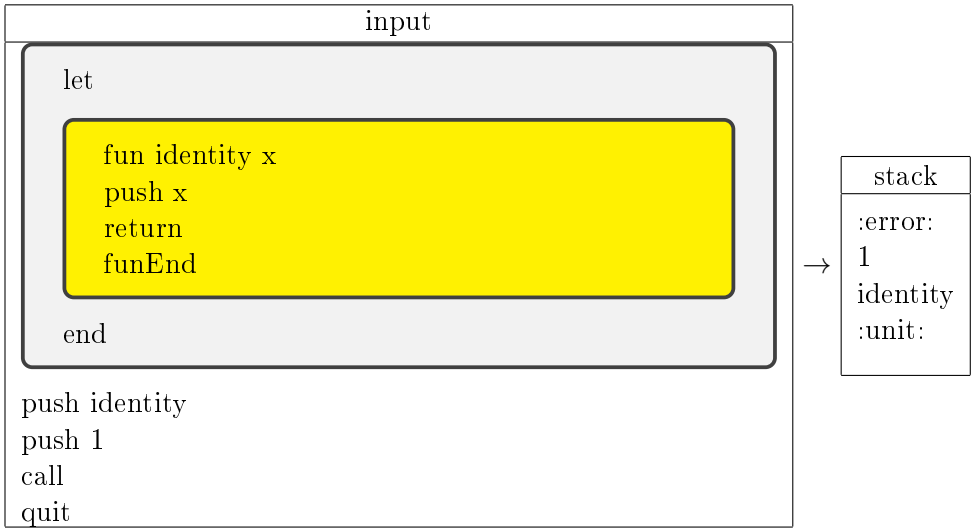


6 → return of calling twiceZ and passing add1 as an argument
:unit: → declaration of twiceZ
:unit: → binding of z
:unit: → declaration of the add1 function

7.4 Functions and Let

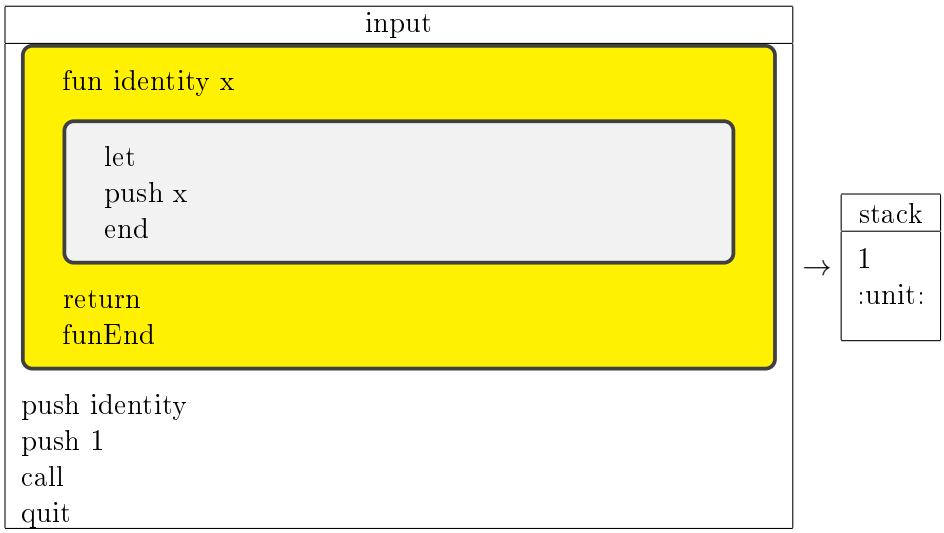
Functions can be declared inside a let expression. Much like the lifetime of a variable binding, the binding of a function obeys the same rules. Since let introduces a stack of environments, the closure should also take this into account. The easiest way to implement this is for the closure to store the stack of environments present at the declaration of the function. (Note: you can create a more optimal implementation by only storing the bindings of the free variables used in the function—to do this you would look up each free variable in the current environment and add a binding from the free variable to the value in the environment stored in the closure)
(please note background color is used only to improve readability):

7.4.1 Example 1



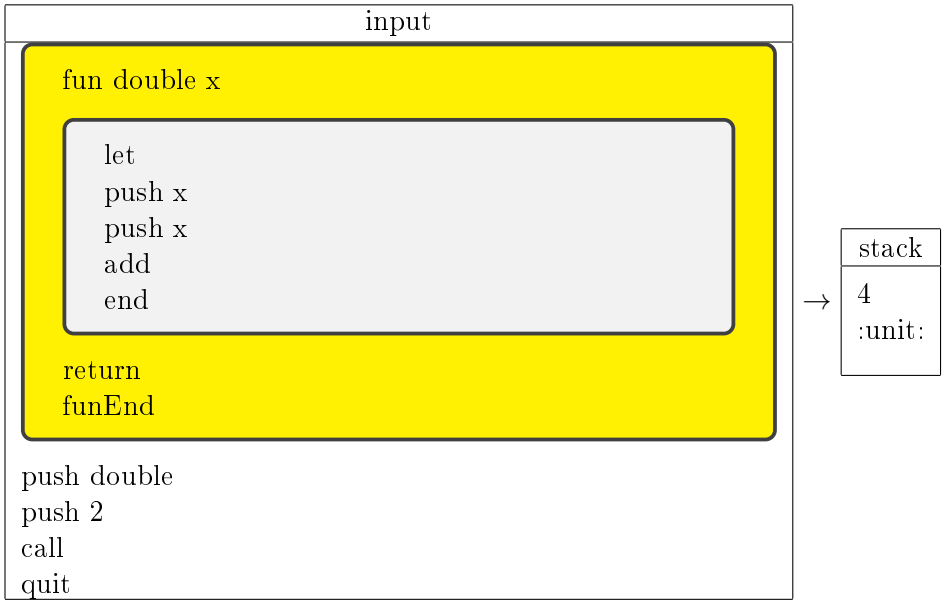
:error: → error since identity is not bound in the environment
1 → push of 1
identity → push of identity
:unit: → result of declaring identity, this is the result of the let expression

7.4.2 Example 2



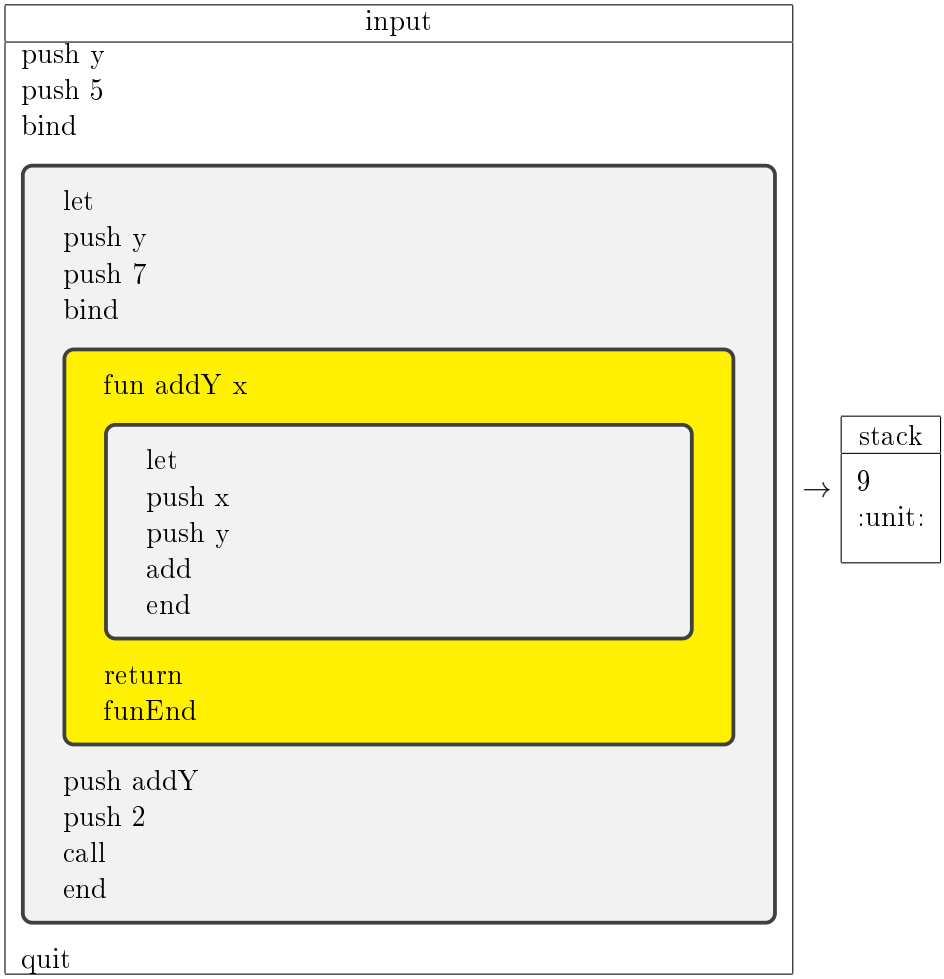
1 → return value of calling identity and passing in x as an argument
:unit: → result of declaring identity

7.4.3 Example 3



4 → return value of calling identity and passing in x as an argument
:unit: → result of declaring identity

7.4.4 Example 4

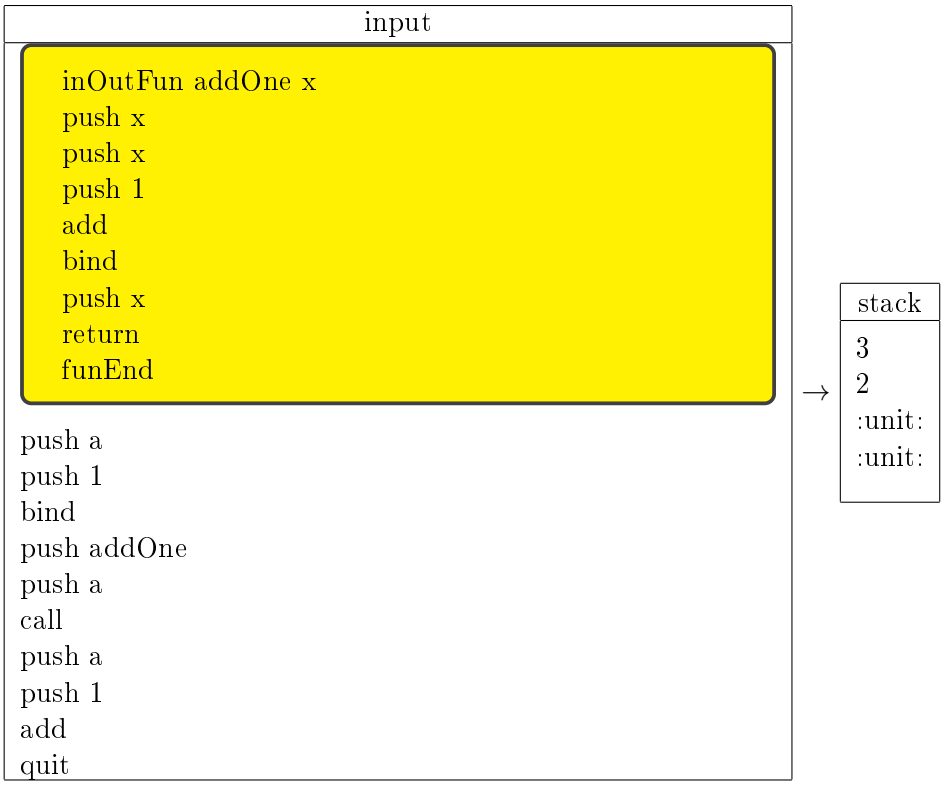


9 → return value of calling identity and passing in 2 as an argument
:unit: → result of binding y to 5

7.5 In/Out Functions

Our language will also support in/out parameters for specially denoted functions. Instead of using the fun keyword, functions that have in/out parameters are declared using the inOutFun keyword. In/out functions behave just like regular functions and all the rules defined for functions apply. In addition, when an in/out function returns, the value bound to the formal parameter is bound to the actual parameter in the environment after the call.

In/out functions should have a similar implementation to regular functions. To this implementation you should add an additional operation when the function returns. In addition to restoring the environment at the call site, the return will do a look up of formal parameter in the environment for the function. This value will be bound to the actual parameter in the environment at the call site.



3 → result of add (note a is bound to two)
2 → return value of calling addOne and passing in x as an argument
:unit: → result of binding a
:unit: → result of declaring addOne

7.6 First-Class Functions

This language treats functions like any other value. They can be used as arguments to functions, and can be returned from functions.

7.6.1 Example 1: Curried adder

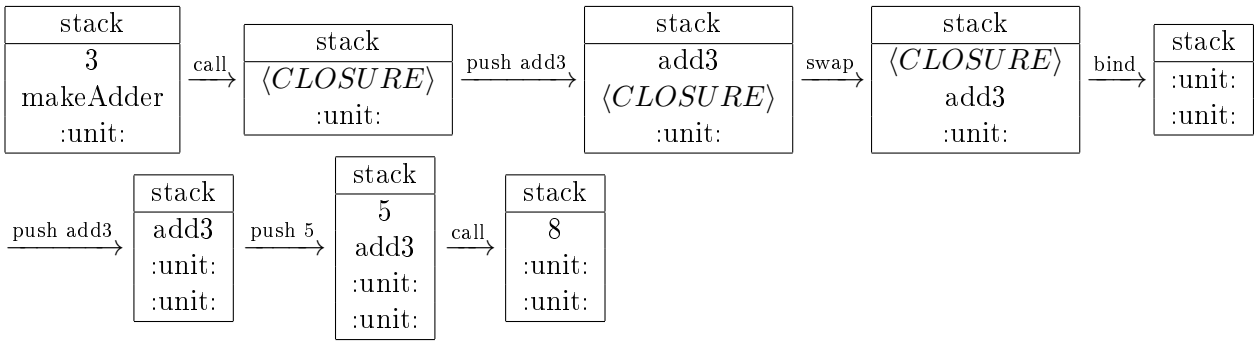
```
input
fun makeAdder x
    fun adder y
    push x
    push y
    add
    return
    funEnd
push adder
return
funEnd
push makeAdder
push 3
call
push add3
swap
bind
push add3
push 5
call
quit
```

→

stack
8
:unit:
:unit:

8 → Evaluated from calling the generated function add3 with argument 5
:unit: → The result of binding the generated function to the name add3
:unit: → The result of declaring the function makeAdder

Step by step (after declaring makeAdder, pushing 3, and pushing makeAdder):



If a function is returned from another function, it need not be bound to a name in the environment it is returned in. For example:

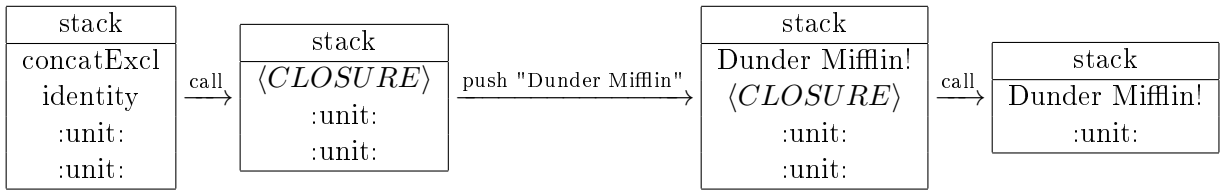
input
fun identity x push x return funEnd fun _catExcl y push y push "!" cat return funEnd push identity push _catExcl call push "Dunder Mifflin" call quit

→

stack
Dunder Mifflin! :unit: :unit:

Dunder Mifflin! → Computed from calling the *closure* returned by the identity function applied to concatExcl with the argument "Dunder Mifflin".
:unit: → The result of declaring the function _catExcl.
:unit: → The result of declaring the identity function.

Here is a closer look at how the stack develops through this program. Note that function closures will never be on the stack when the program finishes execution.



1. You can make the following assumptions:

   - Expressions given in the input file are in correct formats. For example, there will not be expressions like "push", "3" or "add 5" .
   - No multiple operators in the same line in the input file. For example, there will not be "pop pop swap", instead it will be given as

     pop
     pop
     swap

   - No function closures will be left on the stack.
   - All `let` commands will have a matching `end`.

2. You can assume that all test cases will have a quit statement at the end to exit your interpreter, and that "quit" will never appear mid-program.

3. You can assume that your interpreter function will only be called ONCE per execution of your program.

Step by step examples

1. If your interpreter reads in expressions from *inputFile*, states of the stack after each operation are shown below:

input
push 10 push 15 push 30 sub push :true: swap add pop neg quit

First, push 10 onto the stack:

stack
10

Similarly, push 15 and 30 onto the stack:

stack
30 15 10

sub will pop the top two values from the stack, calculate 15 - 30 = -15, and push -15 back:

stack
-15 10

Then push the boolean literal :true: onto the stack:

stack
:true: -15 10

40

swap consumes the top two values, interchanges them and pushes them back:

stack
-15 :true: 10

add will pop the top two values out, which are -15 and :true:, then calculate their sum. Here, :true: is not a numeric value therefore push both of them back in the same order as well as an error literal :error:

stack
:error: -15 :true: 10

pop is to remove the top value from the stack, resulting in:

stack
-15 :true: 10

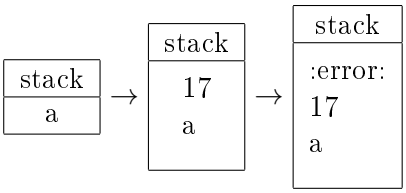
Then after calculating the negation of -15, which is 15, and pushing it back, quit will terminate the interpreter and write the following values in the stack to *outputFile*:

stack
15 :true: 10

Now, go back to the example inputs and outputs given before and make sure you understand how to get those results.

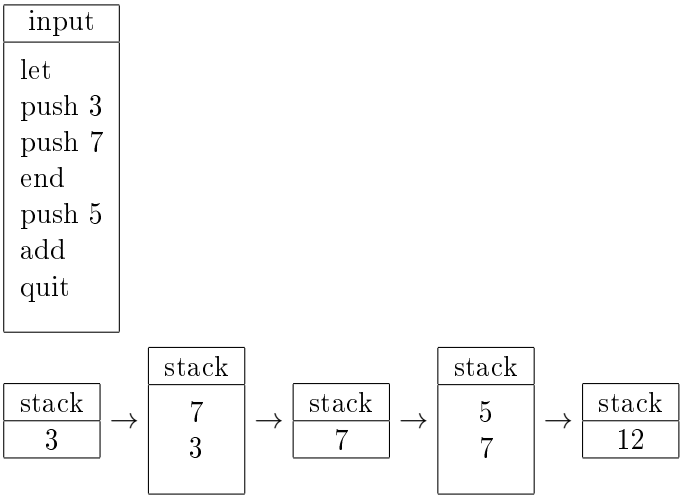
2. More Examples of bind and let...end:

input
push a push 17 add



The error is because we are trying to perform an addition on an unbound variable "a".

3.



Explanation :

push 3
push 7

Pushes 3 and 7 on top of the stack. When you encounter the "end", the last stack frame is saved (which is why the value of 7 is retained on the stack), then 5 is pushed onto the stack and the values are added.

8 Frequently Asked Questions

1. Q: What are the contents of test case X?

   A: We purposefully withhold some test cases to encourage you to write your own test cases and reason about your code. You cannot test *every* possible input into the program for correctness. We will provide high-level overviews of the test cases, but beyond that we expect you to figure out the functionalities that are not checked with the tests we provide. But you can (and should) run the examples shown in this document! They're useful on their own, and can act as a springboard to other test cases.

2. Q: Why does my program run locally but fail on GradeScope?

   A: Check the following:

   - Ensure that your program matches the types and function header defined in section 2 on page 1.
   - Make sure that any testing code is either removed or commented out. If your program calls interpreter with input "input.txt", you will likely throw an exception and get no points.
   - *Do not submit testing code.*

- If you submit a zip or tar file, the main interpreter file must be at the top level of the archive. When extracted, there must be a file named `interpreter.ml` in the current directory.
- `stdout` and `stderr` streams are not graded. Your program must write to the output file specified by *outputFile* for you to receive points.
- *Close your input and output files.*
- Core and any other external libraries are not available.
- GradeScope only supports 4.05, so any features added after are unsupported.

3. Q: Why doesn't GradeScope give useful feedback?

   A: GradeScope is strictly a grading tool to tell you how many test cases you passed and your total score. Test and debug your program locally before submitting to GradeScope. The only worthwhile feedback GradeScope gives is whether or not your program compiled properly.

4. Q: Are there any runtime complexity requirements?

   A: Although having a reasonable runtime and space complexity is important, the only official requirement is that your program runs the test suite in less than three minutes.

5. Q: Is my final score the highest score I received of all my submissions?

   A: No. Your final score is only your most recent submission.

6. Q: What can I do if an old submission received a better grade than my most recent submission?

   A: You can always download any of your previous submissions. If the deadline is approaching, we suggest resubmitting your highest-scoring submission before GradeScope locks.